Практическая работа: Data Placement Map

Длительность: 90 минут
Формат: работа в мини-группах (2 человека) + обсуждение
Артефакты на выходе: черновик Data Placement Map, список спорных решений, Git Issue «Storage architecture v1»
Доступные хранилища: PostgreSQL, Redis, MongoDB, ClickHouse (можно аргументированно предложить другие)

Этап 1. Постановка задачи

Кейс: образовательная платформа «Курсатор»
Платформа для онлайн-обучения. Пользователи проходят курсы, сдают тесты, общаются на форуме. Есть авторы курсов, модераторы, аналитика.

Сущности и потоки данных:

№	Сущность / поток	Операции	Особенности
1	Пользователи	регистрация, вход, редактирование профиля, проверка прав	нужна целостность, связи с ролями
2	Сессии и токены	создание, проверка, истечение по TTL	очень частый доступ, короткая жизнь
3	Курсы и уроки	создание автором, редактирование, публикация	вложенная структура: курс → модули → уроки → материалы; разные курсы имеют разный набор полей
4	Прогресс по курсам	отметка пройденных уроков, результаты тестов	много записей, быстрое чтение текущего состояния
5	Поиск по курсам	полнотекстовый поиск по названию и описанию, фильтры	поиск с опечатками, ранжирование
6	Форум / обсуждения	создание тем, ответы, лайки	древовидная структура, moderate volume
7	Логи действий	поток событий: клики, просмотры уроков, старт/завершение тестов	огромный объём записи, редкие чтения
8	Аналитика и отчёты	«сколько пользователей прошли курс за месяц», «средний процент завершения», «воронка по урокам»	тяжёлые агрегаты по историческим данным
9	Рекомендации курсов	«похожие курсы», «популярное у похожих пользователей»	связи между пользователями и курсами

Задача группе: распределить все 9 сущностей/потоков между хранилищами. Для каждой сущности указать: операции, характер нагрузки, выбранное хранилище, обоснование, спорные моменты.

Этап 2. Анализ сущностей

Каждая группа заполняет аналитическую таблицу — до выбора хранилища. Ответить на вопросы по каждой сущности:

Сущность	Часто читают или пишут?	Нужны транзакции ?	Нужны JOIN / связи?	Объём данных?	Требования к задержке?	Гибкость схемы?
1. Пользователи						
2. Сессии и токены						
3. Курсы и уроки						
4. Прогресс по курсам						
5. Поиск по курсам						
6. Форум / обсуждения						
7. Логи действий						
8. Аналитика и отчёты						
9. Рекомендации курсов						

Вопросы для обсуждения внутри группы:

- Где данные создаются впервые? (источник истины)
- Какие данные можно кэшировать? С каким TTL?
- Какие данные нужны для аналитики? Это те же данные или производные?
- Где нужны ACID-гарантии, а где достаточно eventual consistency?

Этап 3. Выбор хранилищ и заполнение Data Placement Map

Каждая группа заполняет основную таблицу:

Сущность / поток	Операции	Характер нагрузки	Хранилище	Обоснование (2–3 фразы)	Источник истины?	Кэш?	Спорные моменты
1. Пользователи							
2. Сессии и токены							
3. Курсы и уроки							
4. Прогресс по курсам							
5. Поиск по курсам							
6. Форум / обсуждения							
7. Логи действий							
8. Аналитика и отчёты							
9. Рекомендации курсов							

Правила заполнения:

- Одну сущность можно распределить в несколько хранилищ.
- Обязательно указать, где источник истины, а где производные данные.
- Спорные моменты вынести в отдельный список — не пытаться «загладить» неуверенность.

Контрольные вопросы для самопроверки:

1. Есть ли сущность, которую мы положили в Redis, но которой нужны транзакции?
2. Есть ли сущность, которую мы положили в PostgreSQL, но которая растёт без лимита (логи)?
3. Есть ли аналитический запрос, который идёт в OLTP-базу?
4. Есть ли сущность в нескольких хранилищах — и мы не описали, как они синхронизируются?

Этап 4. Синхронизация и потоки данных

Группа рисует **схему потоков данных** — текстовую или в виде простой диаграммы. Ответить на вопросы:

- Что является источником истины для каждой сущности?
- Как данные попадают из источника в кэш / витрину / поисковый индекс?
- Что происходит при обновлении: как узнают производные хранилища?

Шаблон для заполнения:
[сущность] → [хранилище-источник] → [какой механизм] → [производное хранилище]
[сущность] → [хранилище-источник] → [какой механизм] → [производное хранилище]
…

Этап 5. Защита и обсуждение

Каждая группа представляет:

- Свою Data Placement Map (2 минуты)
- 1–2 спорных решения (1 минута)

Цель — не найти единственно верный ответ, а показать, что решения осознанные и аргументированные.

Этап 6. Оформление артефактов

Каждая группа оформляет три артефакта:

1. Data Placement Map — итоговая таблица (из этапа 3), очищенная от черновиков.
2. Список спорных решений — отдельным документом в формате:

```
1. [сущность]: [вариант A] vs [вариант B]
   Аргументы за [A]: …
   Аргументы за [B]: …
   Решение: …

2. …
```

3. **Git Issue «Storage architecture v1»** — краткое сообщение для репозитория:

```
Title: Storage architecture v1 — Data Placement Map

## Решения
- [сущность]: [хранилище] + [доп. хранилище если есть]
- …

## Открытые вопросы
1. …
2. …
3. …

## Следующие шаги
- …
- …
```

Критерии оценки

Критерий	Что проверяется
Соответствие операций и хранилища	Нет антипаттернов
Обоснованность	Каждое решение имеет 2–3 аргумента со ссылкой на критерии (ACID, скорость, объём, поиск)
Источник истины	Для каждой сущности явно указано, где создаётся и обновляются данные
Синхронизация	Описано, как производные хранилища узнают об изменениях
Спорные решения	Выписаны честно, с аргументами «за» и «против»
Артефакты	Data Placement Map, список спорных решений, Git Issue оформлены и сданы